

A2A Protocol

Agent2Agent — Open Standard for Cross-Framework Agent Collaboration

Introduction

A2A (Agent2Agent) is an open protocol, introduced by Google, that standardises how independent AI agents — potentially built by different teams, on different frameworks, in different languages — discover each other, advertise what they can do, and exchange work. Where MCP standardises how a single agent talks to tools and data, A2A standardises how multiple agents talk to each other as peers: one agent can publish an Agent Card describing its skills, another can discover it through a registry, send it a Task wrapped in a Message, and track that task through a defined lifecycle until results come back as artifacts.

A2A's defining idea is treating agent collaboration itself as a protocol problem, not an implementation detail. By agreeing on common building blocks — Agent Cards for capability discovery, a shared Task lifecycle (submitted -> working -> input-required -> completed), and Messages composed of typed Parts (text, data, files) — organisations can mix and match agents from different vendors and frameworks into one cooperating system, in the same way that HTTP lets unrelated web servers and browsers interoperate.

Components

Component	Description
Agent Card	A structured manifest (name, endpoint, capabilities, skills) that describes what an agent can do.
Skill	A single named capability with a description and tags, used for capability-based discovery.
Agent Registry	A directory that stores registered Agent Cards and supports lookup/discovery by skill tag.
Task	The unit of work exchanged between agents; has a unique ID, a description, a state, and artifacts.
Task Lifecycle (state)	submitted -> working -> input-required -> completed (or failed) — the standard progression of a task.
Message	An envelope with a sender, a receiver, and one or more Parts, used to communicate between agents.
Part (TextPart / DataPart / FilePart)	The three basic content types a Message can carry: plain text, structured data, or file references.
Router (skill-based)	Logic that matches an incoming task's description against registered agents' skill tags to pick the best agent.
Artifacts	Files or structured outputs a task produces once completed.
Planner / Orchestrator Agent	An agent that coordinates several specialist agents and aggregates their results into one outcome.

Syntax

```
# 1. Define a Skill and an Agent Card
from pydantic import BaseModel
from typing import List

class Skill(BaseModel):
    id: str
    description: str
    tags: List[str]

class AgentCard(BaseModel):
    name: str
    endpoint: str
    skills: List[Skill]

agent = AgentCard(name="MyAgent", endpoint="http://localhost:8001",
                  skills=[Skill(id="docker", description="...", tags=["docker", "container"])]))

# 2. Register and discover agents
class AgentRegistry:
    def __init__(self): self.agents = []
    def register(self, agent): self.agents.append(agent)
    def discover(self, tag):
        return [a.name for a in self.agents for s in a.skills if tag in s.tags]

# 3. Create and progress a Task through its lifecycle
import uuid
class Task:
    def __init__(self, description):
        self.id = str(uuid.uuid4())
        self.description = description
        self.state = "submitted"
        self.artifacts = []
    def update(self, new_state):
        self.state = new_state

task = Task("Deploy Payment Service")
task.update("working")
task.update("completed")

# 4. Build a multi-part Message between two agents
class Message:
    def __init__(self, sender, receiver):
        self.sender, self.receiver, self.parts = sender, receiver, []

msg = Message("Planner", "Executor")
msg.parts.append(TextPart("Deploy Application"))
msg.parts.append(DataPart({"env": "prod"}))
```

Learning Outcomes

- Design an Agent Card that accurately advertises an agent's skills and capabilities.
- Build an Agent Registry supporting registration and skill-tag-based discovery.
- Model a Task and progress it correctly through the A2A lifecycle states.
- Construct multi-part Messages (text, data, file) for agent-to-agent communication.
- Implement simple skill-based routing to select the best agent for an incoming task.
- Combine Agent Cards, a Registry, Tasks, and multiple specialist agents into one orchestrated A2A system.

Software Requirements

Requirement	Detail
Python Version	3.10 or higher
Core Packages	pydantic>=2.0.0 (schema validation for Agent Cards/Skills)
Other Libraries	rich>=13.0.0 (console output), fastapi>=0.115.0 and uvicorn>=0.30.0 (for exposing agents over HTTP), requests>=2.32.0
LLM Backend	Not required — these labs model the A2A protocol itself with plain Python, independent of any LLM
API Key	None required
Environment	Any local Python environment, terminal, or Jupyter/VS Code setup
Install Command	pip install -r requirements4.txt

Lab Programs

Lab 1: Agent Card

Source Code

```

from pydantic import BaseModel
from typing import List
import json

# =====
# SKILL MODEL
# =====

class Skill(BaseModel):
    id: str
    description: str
    tags: List[str]

# =====
# AGENT CARD MODEL
# =====

class AgentCard(BaseModel):
    name: str
    description: str
    version: str
    endpoint: str
    provider: str
    capabilities: List[str]
    skills: List[Skill]

# =====
# CREATE AGENT CARD
# =====

```

```

infra_agent = AgentCard(
    name="InfrastructureAgent",
    description="Handles server, cloud and deployment operations",
    version="1.0.0",
    endpoint="http://localhost:8001",
    provider="A2A Training Lab",
    capabilities=[
        "Task Execution",
        "Monitoring",
        "Artifact Generation"
    ],
    skills=[
        Skill(
            id="server_health",
            description="Checks server health",
            tags=["server", "health", "monitoring"]
        ),
        Skill(
            id="docker",
            description="Docker container management",
            tags=["docker", "container", "deployment"]
        ),
        Skill(
            id="autoscaling",
            description="Scale infrastructure resources",
            tags=["cloud", "scaling", "aws"]
        )
    ]
)

# =====
# DISPLAY CARD
# =====

print("\nAGENT CARD")
print("=" * 50)
print(infra_agent.model_dump_json(indent=4))

```

Sample Input / Output

```

infra_agent = AgentCard(name="InfrastructureAgent", ..., skills=[Skill(id="docker", ...)])

--- Output ---
AGENT CARD
=====
{
  "name": "InfrastructureAgent",
  "description": "Handles server, cloud and deployment operations",
  "version": "1.0.0",
  "endpoint": "http://localhost:8001",
  "provider": "A2A Training Lab",
  "capabilities": ["Task Execution", "Monitoring", "Artifact Generation"],
  "skills": [
    {"id": "server_health", "description": "Checks server health", "tags":
["server","health","monitoring"]},
    {"id": "docker", "description": "Docker container management", "tags":
["docker","container","deployment"]},
    {"id": "autoscaling", "description": "Scale infrastructure resources", "tags":
["cloud","scaling","aws"]}
  ]
}

```

Key Concepts Covered

Concept	Description
Agent Card	A self-describing manifest (name, endpoint, capabilities, skills) that announces what an agent can do.
Skill	A single named capability with a description and searchable tags, nested inside an Agent Card.
Pydantic BaseModel	Used to define and validate the structured Agent Card / Skill schema.

Lab 2: Agent Registry

Source Code

```

from pydantic import BaseModel
from typing import List

print("Program Started")

class Skill(BaseModel):
    id: str
    description: str
    tags: List[str]

class AgentCard(BaseModel):
    name: str
    endpoint: str
    skills: List[Skill]

class AgentRegistry:

    def __init__(self):
        self.agents = []

    def register(self, agent):
        self.agents.append(agent)

    def discover(self, tag):
        matches = []
        for agent in self.agents:
            for skill in agent.skills:
                if tag in skill.tags:
                    matches.append(agent.name)
        return matches

infra = AgentCard(
    name="InfrastructureAgent",
    endpoint="http://localhost:8001",
    skills=[
        Skill(id="docker", description="Docker Operations", tags=["docker", "container"])
    ]
)

cloud = AgentCard(
    name="CloudAgent",
    endpoint="http://localhost:8002",
    skills=[
        Skill(id="aws", description="AWS Operations", tags=["aws", "cloud"])
    ]
)

registry = AgentRegistry()
registry.register(infra)
registry.register(cloud)

print("\nRegistered Agents")
for agent in registry.agents:
    print("-", agent.name)

print("\nSearching for 'cloud' skill")
result = registry.discover("cloud")
print("Found:", result)

```

Sample Input / Output

```
registry.register(infra)
registry.register(cloud)
registry.discover("cloud")
```

```
--- Output ---
Registered Agents
- InfrastructureAgent
- CloudAgent
```

```
Searching for 'cloud' skill
Found: ['CloudAgent']
```

Key Concepts Covered

Concept	Description
Agent Registry	A central directory that stores registered Agent Cards for later lookup.
register()	Adds an Agent Card to the registry's known-agents list.
discover(tag)	Finds agents whose skills include a given tag — the core of capability-based discovery.

Lab 3: Task Lifecycle

Source Code

```
import uuid
import time

class Task:

    def __init__(self, description):
        self.id = str(uuid.uuid4())
        self.description = description
        self.state = "submitted"
        self.artifacts = []

    def update(self, new_state):
        self.state = new_state
        print(f"{self.id[:8]} -> {new_state}")

task = Task("Deploy Payment Service")

states = [
    "working",
    "input-required",
    "working",
    "completed"
]

for state in states:
    time.sleep(1)
    task.update(state)

task.artifacts.append("deployment_report.pdf")
print(task.artifacts)
```

Sample Input / Output

```
task = Task("Deploy Payment Service")
for state in ["working", "input-required", "working", "completed"]:
    task.update(state)

--- Output ---
a1b2c3d4 -> working
a1b2c3d4 -> input-required
a1b2c3d4 -> working
a1b2c3d4 -> completed
['deployment_report.pdf']
```

Key Concepts Covered

Concept	Description
Task	A2A's unit of work, identified by a UUID and tracked through a sequence of states.
Task State / Lifecycle	submitted -> working -> input-required -> working -> completed models real-world progress, including pauses for human input.
Artifacts	Files or outputs produced by a task, attached once work is done.

Lab 4: Message Exchange

Source Code

```
class TextPart:

    def __init__(self, text):
        self.text = text

class DataPart:

    def __init__(self, data):
        self.data = data

class FilePart:

    def __init__(self, file):
        self.file = file

class Message:

    def __init__(self, sender, receiver):
        self.sender = sender
        self.receiver = receiver
        self.parts = []

msg = Message("Planner", "Executor")

msg.parts.append(TextPart("Deploy Application"))
msg.parts.append(DataPart({"env": "prod"}))
msg.parts.append(FilePart("deployment.yaml"))

for part in msg.parts:
    if hasattr(part, "text"):
        print("TEXT:", part.text)
    if hasattr(part, "data"):
        print("DATA:", part.data)
    if hasattr(part, "file"):
        print("FILE:", part.file)
```

Sample Input / Output

```
msg = Message("Planner", "Executor")
msg.parts = [TextPart(...), DataPart(...), FilePart(...)]

--- Output ---
TEXT: Deploy Application
DATA: {'env': 'prod'}
FILE: deployment.yaml
```

Key Concepts Covered

Concept	Description
Message	An A2A envelope with a sender, receiver, and a list of typed parts.
Multi-Part Payload	A single message can carry text, structured data, and file references together.

Concept	Description
TextPart / DataPart / FilePart	A2A's three basic content types for message parts, each handled differently by the receiver.

Lab 5: Skill Routing

Source Code

```
class Router:

    def __init__(self):
        self.skills = {
            "DockerAgent": ["docker", "container", "image"],
            "K8sAgent": ["kubernetes", "k8s", "pod"],
            "TerraformAgent": ["terraform", "iac", "infrastructure"]
        }

    def route(self, task):
        scores = {}
        task = task.lower()
        for agent, tags in self.skills.items():
            scores[agent] = sum(tag in task for tag in tags)
        return scores

router = Router()

result = router.route("Deploy container image using docker")
print(result)

winner = max(result, key=result.get)
print("Selected:", winner)
```

Sample Input / Output

```
router.route("Deploy container image using docker")

--- Output ---
{'DockerAgent': 3, 'K8sAgent': 0, 'TerraformAgent': 0}
Selected: DockerAgent
```

Key Concepts Covered

Concept	Description
Skill-Based Routing	Matches an incoming task's text against each agent's known skill tags to find the best fit.
Scoring	Counts tag matches per agent, turning routing into a simple ranking problem.
<code>max(scores, key=scores.get)</code>	Selects the highest-scoring (most relevant) agent for the task.

Lab 6: A2A System — Direct Hand-off & Orchestrated Execution

Source Code — groupchat.py (sequential hand-off)

```
class Agent:

    def __init__(self, name):
        self.name = name

    def respond(self, message):
        return f"{self.name}: handled {message}"

planner = Agent("Planner")
executor = Agent("Executor")
monitor = Agent("Monitor")

history = []

msg = "Deploy Inventory Service"

response = planner.respond(msg)
history.append(response)

response = executor.respond(response)
history.append(response)

response = monitor.respond(response)
history.append(response)

for h in history:
    print(h)
```

Source Code — a2asystem.py (orchestrated execution)

```
import uuid

class Executor:

    def deploy(self):
        return {
            "status": "success",
            "artifact": "deployment.log"
        }

class Monitor:

    def verify(self):
        return {
            "health": "healthy",
            "cpu": "35%"
        }

class Planner:

    def __init__(self):
        self.executor = Executor()
        self.monitor = Monitor()

    def execute(self, task):
        print("Task:", task)

        deploy = self.executor.deploy()
```

```

        verify = self.monitor.verify()

        return {
            "task_id": str(uuid.uuid4()),
            "deployment": deploy,
            "verification": verify,
            "status": "completed"
        }

    planner = Planner()

    result = planner.execute("Deploy Payment Service")

    print(result)

```

Sample Input / Output

```

# Pattern A (groupchat.py): sequential message hand-off
msg = "Deploy Inventory Service"

# Pattern B (a2asystem.py): planner orchestrates executor + monitor
planner.execute("Deploy Payment Service")

--- Output ---
Planner: handled Deploy Inventory Service
Executor: handled Planner: handled Deploy Inventory Service
Monitor: handled Executor: handled Planner: handled Deploy Inventory Service

---

Task: Deploy Payment Service
{'task_id': '7e2f...', 'deployment': {'status': 'success', 'artifact': 'deployment.log'},
 'verification': {'health': 'healthy', 'cpu': '35%'}, 'status': 'completed'}

```

Key Concepts Covered

Concept	Description
Direct Agent-to-Agent Hand-off	groupchat.py chains agents in sequence, each one passing its response to the next (a minimal A2A relay).
Orchestrated Execution	a2asystem.py has a Planner agent that calls an Executor and a Monitor directly and combines both results into one outcome.
Task Result Aggregation	Both patterns show a coordinator combining sub-agent outputs into a single structured result (a task_id + status).

Capstone: Capstone — Multi-Agent Incident Management System (Full A2A)

Source Code — capstonea2a.py

```
import uuid
import json
from datetime import datetime

# =====
# AGENT CARD
# =====

class AgentCard:

    def __init__(self, name, skills):
        self.name = name
        self.skills = skills

# =====
# AGENT REGISTRY
# =====

class AgentRegistry:

    def __init__(self):
        self.agents = {}

    def register(self, agent_card):
        self.agents[agent_card.name] = agent_card

    def list_agents(self):
        print("\nREGISTERED AGENTS")
        for name, card in self.agents.items():
            print(f"{name} -> {card.skills}")

# =====
# MESSAGE
# =====

class Message:

    def __init__(self, sender, receiver, content):
        self.sender = sender
        self.receiver = receiver
        self.content = content

# =====
# TASK
# =====

class Task:

    def __init__(self, description):
        self.task_id = str(uuid.uuid4())
        self.description = description
        self.status = "submitted"
        self.artifacts = []

    def update_status(self, status):
        self.status = status
        print(f"[TASK] {self.task_id[:8]} -> {status}")

# =====
# MONITORING AGENT
```

```

# =====

class MonitoringAgent:

    def handle(self, task):
        print("\n[Monitoring Agent]")
        metrics = {
            "cpu": "98%",
            "memory": "72%",
            "traffic": "High Traffic Spike"
        }
        return metrics

# =====
# INFRASTRUCTURE AGENT
# =====

class InfrastructureAgent:

    def handle(self, task):
        print("\n[Infrastructure Agent]")
        health = {
            "server": "Healthy",
            "database": "Healthy",
            "instances": 2
        }
        return health

# =====
# SECURITY AGENT
# =====

class SecurityAgent:

    def handle(self, task):
        print("\n[Security Agent]")
        security = {
            "threats": "None",
            "ddos": "No",
            "intrusion": "No"
        }
        return security

# =====
# RESOLUTION AGENT
# =====

class ResolutionAgent:

    def handle(self, monitoring, infra, security):
        print("\n[Resolution Agent]")

        if monitoring["cpu"] == "98%":
            return {
                "root_cause": "Traffic Spike",
                "resolution": "Scale instances from 2 to 5",
                "status": "Resolved"
            }

        return {
            "root_cause": "Unknown",
            "resolution": "Manual Investigation",
            "status": "Pending"
        }

# =====

```

```

# PLANNER AGENT (orchestrates all specialist agents)
# =====

class PlannerAgent:

    def __init__(self):
        self.monitor = MonitoringAgent()
        self.infra = InfrastructureAgent()
        self.security = SecurityAgent()
        self.resolution = ResolutionAgent()

    def execute(self, task):
        task.update_status("working")

        monitor_result = self.monitor.handle(task)
        infra_result = self.infra.handle(task)
        security_result = self.security.handle(task)

        resolution_result = self.resolution.handle(
            monitor_result, infra_result, security_result
        )

        task.artifacts.extend([
            "metrics.json",
            "infra_report.json",
            "security_report.json"
        ])

        task.update_status("completed")

        report = {
            "incident": task.description,
            "monitoring": monitor_result,
            "infrastructure": infra_result,
            "security": security_result,
            "resolution": resolution_result,
            "artifacts": task.artifacts,
            "generated_at": str(datetime.now())
        }

        return report

# =====
# MAIN
# =====

registry = AgentRegistry()

registry.register(AgentCard("MonitoringAgent", ["metrics", "cpu", "memory"]))
registry.register(AgentCard("InfrastructureAgent", ["server", "database"]))
registry.register(AgentCard("SecurityAgent", ["threat", "attack"]))
registry.register(AgentCard("ResolutionAgent", ["analysis", "resolution"]))

registry.list_agents()

print("\n=====")
print("INCIDENT MANAGEMENT SYSTEM")
print("=====")

incident = input("\nEnter Incident: ")

task = Task(incident)

planner = PlannerAgent()

```

```

final_report = planner.execute(task)

print("\n=====")
print("FINAL INCIDENT REPORT")
print("=====")

print(json.dumps(final_report, indent=4))

```

Sample Input / Output

Enter Incident: Website is down, high CPU usage reported

--- Output ---

REGISTERED AGENTS

MonitoringAgent -> ['metrics', 'cpu', 'memory']

InfrastructureAgent -> ['server', 'database']

SecurityAgent -> ['threat', 'attack']

ResolutionAgent -> ['analysis', 'resolution']

[TASK] 9f3a2b1c -> working

[Monitoring Agent]

[Infrastructure Agent]

[Security Agent]

[Resolution Agent]

[TASK] 9f3a2b1c -> completed

FINAL INCIDENT REPORT

```

{
  "incident": "Website is down, high CPU usage reported",
  "monitoring": {"cpu": "98%", "memory": "72%", "traffic": "High Traffic Spike"},
  "infrastructure": {"server": "Healthy", "database": "Healthy", "instances": 2},
  "security": {"threats": "None", "ddos": "No", "intrusion": "No"},
  "resolution": {"root_cause": "Traffic Spike", "resolution": "Scale instances from 2 to 5"},
  "status": "Resolved",
  "artifacts": ["metrics.json", "infra_report.json", "security_report.json"],
  "generated_at": "2026-06-25 10:42:11"
}

```

Key Concepts Covered

Concept	Description
Full A2A Pipeline	Combines Agent Card, Agent Registry, Task lifecycle, and multi-agent hand-off into one real workflow.
PlannerAgent as Orchestrator	Calls Monitoring, Infrastructure, and Security agents, then passes all three results to a Resolution agent.
Task Status Tracking	task.update_status() marks the task 'working' then 'completed', mirroring Lab 3's lifecycle states.
Structured Final Report	All sub-agent outputs and artifacts are aggregated into one JSON report — the end deliverable of the A2A workflow.

Cheat Sheet

Syntax / Call	Purpose
AgentCard(name, endpoint, skills, ...)	Define a structured manifest describing an agent's capabilities.
Skill(id, description, tags)	Define a single discoverable capability with searchable tags.
registry.register(agent_card)	Add an agent to the registry so it can be discovered.
registry.discover(tag)	Find all registered agents whose skills include a given tag.
Task(description)	Create a new task with a unique ID and initial state 'submitted'.
task.update(state) / task.update_status(state)	Move a task to its next lifecycle state (working, completed, etc.).
Message(sender, receiver)	Create an envelope for agent-to-agent communication.
msg.parts.append(TextPart/DataPart/FilePart(...))	Attach typed content to a message.
router.route(task_description)	Score each registered agent against a task description by tag overlap.
max(scores, key=scores.get)	Pick the highest-scoring agent from a routing result.

Industry Use Cases

Pattern	Real-World Application
Agent Card / capability discovery	Agent marketplaces, enterprise agent catalogs where teams publish reusable agents.
Agent Registry	Service-discovery layers for multi-agent platforms, similar to microservice registries.
Task lifecycle	Long-running workflow tracking — DevOps pipelines, approval processes, ticketing systems.
Multi-part Messages	Rich agent communication carrying instructions, structured parameters, and file attachments together.
Skill-based routing	Dynamic dispatch in IT operations platforms (route a request to the right specialist agent automatically).
Multi-agent orchestration (capstone)	Incident response/SRE automation, cross-team coordination systems (monitoring + infra + security + resolution).

Troubleshooting

Issue	Solution
pydantic.ValidationError when creating an AgentCard	Check every required field (name, endpoint, skills, etc.) is supplied with the correct type.
discover(tag) returns an empty list	Confirm the tag string matches exactly (case-sensitive) one of the registered skill's tags.
Task stuck showing the wrong state	Make sure update()/update_status() is called in the correct order and not skipped for any branch.
AttributeError when reading a Message part	Use hasattr(part, 'text'/'data'/'file') before accessing, since parts are different types.
Router always picks the same agent	Verify the skills dictionary's tags actually differ meaningfully between agents and match real task keywords.
ModuleNotFoundError: pydantic	Run pip install -r requirements4.txt to install pydantic, rich, fastapi, uvicorn, and requests.

Conclusion

Across these labs, students built up every core primitive of the A2A protocol — Agent Cards for capability discovery, an Agent Registry for lookup, a Task lifecycle for tracking work, multi-part Messages for rich communication, and skill-based routing for automatic dispatch — before combining them all in a capstone multi-agent incident management system. The exercises show how A2A turns agent collaboration from ad-hoc, framework-specific glue code into a consistent, interoperable protocol that scales from a two-agent hand-off to a full multi-specialist orchestration pipeline.